

Начало работы

Быстрый старт

📄 Copy page



Начните работу с Zapier SDK за 5 минут

Настройте свой ИИ

Ваш помощник выполняет команды за вас



Открыть в Codex



Открыть в Claude Code



Открыть в Cursor



Открыть в VS Code
Copilot



Ваш помощник проведет вас через весь процесс



Открыть в ChatGPT



Открыть в Claude



Открыть в Perplexity



Используете другого помощника для IDE (Cline, Windsurf и т. д.)? Скопируйте эту подсказку.

Set up the Zapier SDK for me. Before installing anything:

- Run `pwd` and tell me the current folder.
- Ask whether to install here or somewhere else.

End with: "Should I set up here, or do you want to point me at a different dir

... Посмотреть все 76 строк

Требования

- Установлен Node.js 20+
- Учетная запись Zapier (бесплатный тарифный план). Или создайте ее с помощью интерфейса командной строки на шаге 3
- Хотя бы одно приложение, подключенное к вашей учетной записи Zapier

Шаг 1. Установите SDK

Создайте новый проект (или используйте существующий) и установите необходимые пакеты:

```
# Create a new project (optional)
mkdir my-zapier-project && cd my-zapier-project
npm init -y && npm pkg set type=module

# Install the SDK and CLI
npm install @zapier/zapier-sdk
npm install -D @zapier/zapier-sdk-cli @types/node typescript

# Initialize TypeScript (if starting fresh)
npx tsc --init
```



Примерах из этого руководства предполагается, что у вас есть `"type": "module"` в вашем `package.json` так, чтобы вы использовали модули ES с поддержкой верхнего уровня `await`. Мы сделали это выше с помощью `npm pkg set type=module`.

Шаг 2. Установите навык SDK

Установите навык Zapier SDK, чтобы ваш кодер знал, как использовать SDK в каждом последующем сеансе:

```
npx skills add zapier/sdk -y
```

Работает с Claude Code, Cursor, Codex, OpenCode, Windsurf и более чем 50 другими кодерами.

Шаг 3. Аутентификация

SDK CLI может создать учетную запись Zapier или войти в уже существующую, а затем предоставить локальные учетные данные для SDK:

Впервые пользуетесь Zapier? Начните с `signup` :

```
npx zapier-sdk signup
```

Откроется ваш браузер для создания учетной записи, авторизации в командной строке SDK и сохранения локальных учетных данных SDK.

У вас уже есть учетная запись Zapier? Используйте `login` вместо этого:

```
npx zapier-sdk login
```

Пока пакет CLI установлен в качестве зависимости для разработки, SDK будет автоматически использовать вашу аутентификацию через CLI.



Building for production or deploying to a server? CLI authentication is for development terminals. For any server-side deployment, client credentials are the recommended approach.

- ❗ **Want approvals on these credentials?** Pass `--use-approvals` to `signup` or `login` to provision the credential with an empty permission policy. Every request will prompt you to approve (or deny) the action until you've built the policy up. See Permissions for the full flow.

```
npx zapier-sdk signup --use-approvals
# or, if you already have an account:
npx zapier-sdk login --use-approvals
```

Alternative: Headless authentication

Alternative: Client Credentials

Alternative: Direct Token

Step 4: Generate Types for Your Apps

The SDK can generate TypeScript types for any app, giving you full autocomplete and type safety:

```
# Add types for the apps you want to use
npx zapier-sdk add slack google-sheets

# Don't know the app key? Search for it
npx zapier-sdk list-apps --search "google sheets"
```

По умолчанию типы генерируются в папке `src` или `lib`. Вы можете изменить расположение выходных данных:

```
npx zapier-sdk add slack --types-output ./types
```

Step 5: Initialize the SDK

Create a new file (e.g., `index.ts`) and initialize the SDK:

```
import { createZapierSdk } from "@zapier/zapier-sdk";

// Initialize with browser-based auth (from `zapier-sdk signup` or `zapier-sdk`
const zapier = createZapierSdk();
```

Step 6: List Your Connected Apps

Let's verify everything works by listing available apps:

```
// List the first page of apps
const { data: apps } = await zapier.listApps();

console.log(
  "Available apps:",
  apps.map((app) => app.title)
);
```

💡 This returns the first page of results. For large datasets, use `.items()` to iterate over all results or `maxItems` to limit the total. See the [API Reference](#) for pagination patterns.

Запустите скрипт:

```
npx tsx index.ts
```

Step 7: Run Your First Action

Now let's execute an action. First, you'll need a connection for the app you want to use:



```
// Get Slack connection
```

```
// Option 1: Use listConnections when you need to filter or  
// work with multiple connections.
```

```
const { data: allSlackConnections } = await zapier.listConnections({  
  appKey: "slack",  
  owner: "me",  
  isExpired: false,  
});  
  
const acmeSlackConnection = allSlackConnections.find(c => c?.title?.toLowerCase === "acme")  
if (!acmeSlackConnection) {  
  console.log(  
    "Slack connection matching filter not found. Connect Slack at https://zapier.com/apps/slack/integrations/  
  );  
}
```

```
// Option 2: Use findFirstConnection when you just need the first  
// available connection and let any errors bubble up
```

```
const { data: firstSlackConnection } = await zapier.findFirstConnection({  
  appKey: "slack",  
  owner: "me",  
});
```

```
// List Slack channels using your connection
```

```
const { data: channels } = await zapier.runAction({  
  appKey: "slack",  
  actionType: "read",  
  actionKey: "channels",  
  connectionId: firstSlackConnection.id,  
});
```

```
console.log("Your Slack channels:", channels);
```

Step 8: Use the Proxy Pattern (Optional)

For a cleaner syntax, use the app proxy pattern:



```
const { data: firstSlackConnection } = await zapier.findFirstConnection({
  appKey: "slack",
  owner: "me",
  isExpired: false,
});

// Create a bound Slack instance
const mySlack = zapier.apps.slack({
  connectionId: firstSlackConnection.id,
});

// Now use it with a clean syntax
const { data: channels } = await mySlack.read.channels({});

console.log("Channels:", channels);

// Or pass auth inline without binding
const { data: users } = await zapier.apps.slack.search.user_by_email({
  inputs: { email: "colleague@company.com" },
  connectionId: firstSlackConnection.id,
});
```

Step 9: Make Custom API Calls with `fetch` (Optional)

- ❗ **Note on governance:** The `.fetch()` method makes authenticated API calls directly, giving you access to any supported endpoint. Unlike pre-built actions, these direct API calls are not currently subject to your org's app or action restriction policies. If your org has governance requirements, use pre-built actions where possible. Direct API governance is on the roadmap.

Если вам нужно вызвать API, для которого нет встроенного действия, используйте `fetch` для отправки аутентифицированных запросов через Zapier SDK:



```
// Get your Slack connection
```

```
const { data: slackConnection } = await zapier.findFirstConnection({
  appKey: "slack",
  owner: "me",
  isExpired: false,
});

if (!slackConnection) {
  console.log(
    "No Slack connection found. Connect Slack at https://zapier.com/app/assets
  );
  process.exit(1);
}

// Make a custom API call—Zapier injects the user's credentials
const response = await zapier.fetch("https://slack.com/api/users.list", {
  method: "GET",
  connectionId: slackConnection.id,
});

const users = await response.json();
```

Complete Example

Here's a full example that sends a Slack message:


```
import { createZapierSdk } from "@zapier/zapier-sdk";
```

```
async function main() {
  const zapier = createZapierSdk();

  // Get Slack connection
  const { data: firstSlackConnection } = await zapier.findFirstConnection({
    appKey: "slack",
    owner: "me",
    isExpired: false,
  });

  // Create bound Slack instance
  const slack = zapier.apps.slack({
    connectionId: firstSlackConnection.id,
  });

  // Get available channels
  const { data: channels } = (await slack.read.channels({})) as {
    data: Array<{ id: string; name: string }>;
  };
  const testChannel = channels.find((c) => c.name === "testing");

  if (!testChannel) {
    throw new Error("Could not find #testing channel");
  }

  // Send a message
  const { data: result } = await slack.write.channel_message({
    inputs: {
      channel: testChannel.id,
      text: "Hello from the Zapier SDK!",
    },
  });

  console.log("Message sent!", result);
}

main().catch(console.error);
```

Next Steps

- [API Reference](#) — full documentation of all SDK methods and patterns
 - [CLI Reference](#) — command-line tools including list-apps, add, and view-policy
-
- [Browse integrations](#) — 9,000+ apps you can connect
 - [Try the demo use case](#) — the meeting reschedule example on the overview page is a confirmed working end-to-end scenario, good for testing your setup
 - [Hit a problem?](#) — tell us what broke via the feedback form

Last modified on August 3, 2026

Was this page helpful?

 Yes

 No

[◀ Zapier SDK](#)

[Using the CLI ▶](#)